PART II

Event Sourcing, CQRS & Event-Driven Microservices Architecture

A Hands-On Deep Dive into the Event Store, Eventual Consistency, Event Hubs, and Real-Time Stream Processing — with Full Java / Spring Boot Implementations

Java 17 | Spring Boot 3.x | Apache Kafka | Kafka Streams | Apache Spark | SQL Server | Cassandra | Kubernetes / HPA

Table of Contents

1. How This Part Connects to Part I 3
2. Event Sourcing Pattern — Definition & Deep Dive 4
3. Hands-On: Building an Event-Sourced Order Service 6
4. CQRS + Event Sourcing + Eventual Consistency — The Combined Architecture 10
5. Hands-On: SQL Server Write Side + Cassandra Read Side + Kafka Sync 12
6. Event Hub — Definition & Deep Dive 15
7. Hands-On: Kafka as an Event Hub for Multiple Consumer Types 16
8. Event-Driven Microservices Architecture — The Full Picture 18
9. Hands-On: Apache Spark Structured Streaming on the Event Hub 19
10. Capacity Planning: Concurrency, Containers & Orchestrators 21
11. Recap & Where to Go Next 23

1. How This Part Connects to Part I

Part I of this guide walked through ten core microservice design patterns end to end, including a first pass at CQRS (Pattern 2) and Event Sourcing (Pattern 3) using PostgreSQL and Cassandra. Part II goes one level deeper on exactly that thread — Event Sourcing, CQRS, Eventual Consistency, Event Hubs, and the fully event-driven architecture they compose into — following the same e-commerce Order domain, but adding two pieces of tech stack Part I didn't cover: **Microsoft SQL Server** as the write-side relational store, and **Apache Spark Structured Streaming** for real-time stream processing on top of the event backbone.

Think of Part II as answering one specific question in full depth: *once every service in the system is publishing and consuming events, how far can this architecture actually scale, and what does the code that makes it work really look like?*

> **Quick recap of the core idea:** instead of a database row holding only the *current* state of an order, an event-sourced system holds every state transition — `OrderCreated`, `ItemAdded`, `ItemRemoved`, `OrderConfirmed` — as an immutable, ordered record. Current state is derived, not stored. That single shift in perspective is what unlocks CQRS, reliable eventual consistency, and a natural event backbone for the rest of the platform.

Concept	Covered in Part I	Extended in Part II
Event Sourcing	Cassandra event store, snapshots, replay	Full aggregate lifecycle, optimistic concurrency, upcasting old event versions
CQRS	PostgreSQL write / Cassandra read	SQL Server write / Cassandra read, explicit eventual-consistency SLAs and read-your-writes handling
Event backbone	Kafka topics per bounded context	Kafka as a platform-wide Event Hub serving many independent consumer types simultaneously
Stream processing	Not covered	Apache Spark Structured Streaming reacting to the event hub in near real time
Capacity planning	A simple table (Section 1)	Concrete Kubernetes HPA config tied to request-per-second targets

2. Event Sourcing Pattern — Definition & Deep Dive

The Core Idea

Traditional persistence saves only the *latest* status of a record — every update overwrites whatever was there before. The Event Sourcing pattern changes what gets saved: rather than updating a row in place, every change to an entity is appended as a new, immutable event to a strictly ordered sequence. The database that holds this sequence is called the **event store**, and it becomes the single source of truth for that entity — not a cache of it, not a projection of it, *the truth itself*. Current state is never stored directly; it is produced on demand by replaying the event sequence from the beginning (or from the last snapshot, as covered in Part I).

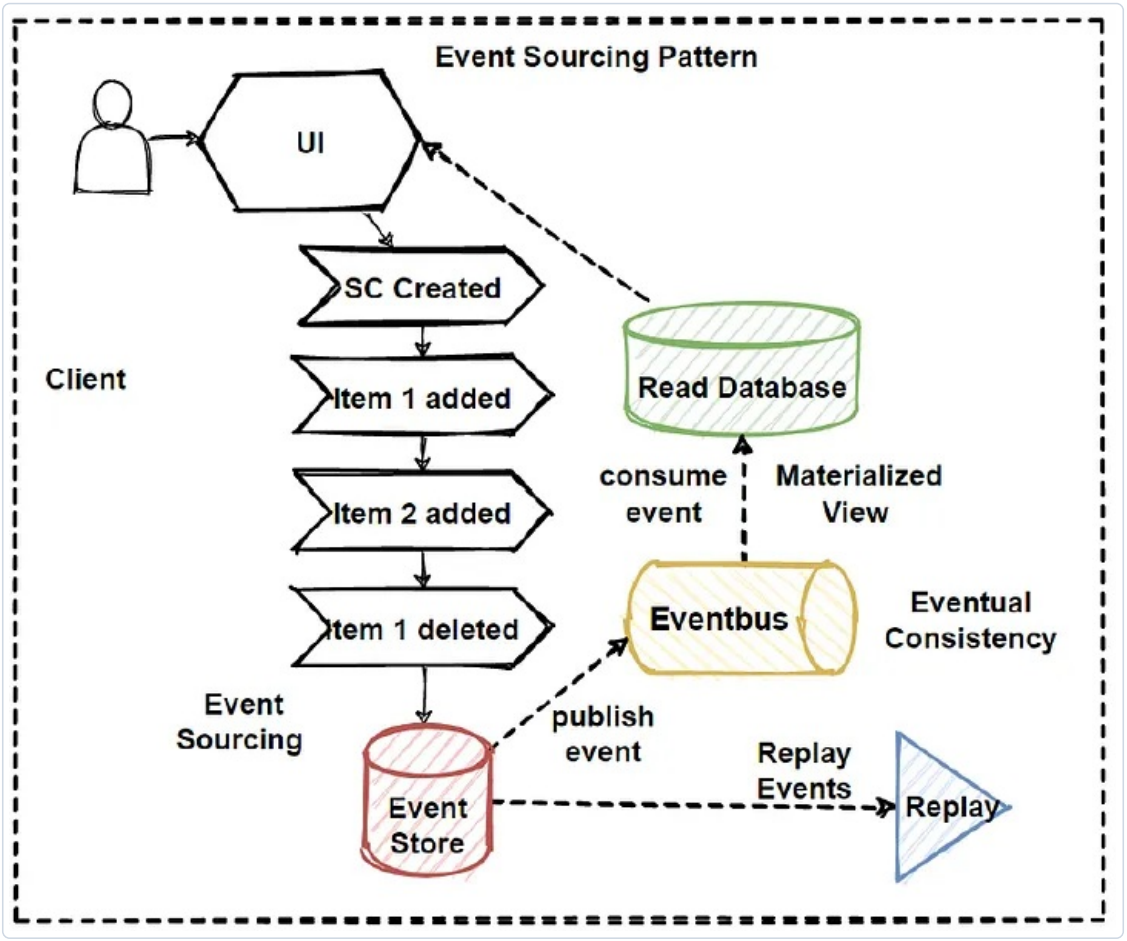


Fig. 1 — The Event Sourcing lifecycle: every UI action (SC Created, Item 1 added, Item 2 added, Item 1 deleted) is appended to the Event Store. Events are published onto an Eventbus, consumed to build a Materialized View in the Read Database, and the UI reads back from that view — while the full event history remains available for Replay at any time.

Why Append-Only Beats Update-in-Place

Traditional (update-in-place)	Event Sourcing (append-only)
Only the current value survives; history is lost unless a separate audit table is maintained	The full history is the data model itself — nothing extra needs to be built for audit trails
Debugging "how did we get into this state" requires reconstructing history from logs, if they still exist	Replaying events reproduces the exact sequence of state transitions, on demand, forever
A bug in a derived read model requires a data migration to fix retroactively	A bug in a projector can be fixed and the read model rebuilt from scratch by replaying every event again
Concurrent updates typically rely on row-level locks or last-write-wins	Concurrent writes are naturally modeled as competing events on the same aggregate, checked with optimistic concurrency on the next expected sequence number

From Event Store to Read Model: The Conversion Step

An event store on its own is not queryable in the way a UI needs — you can't easily ask "give me every order over $500 placed this month" by scanning an append-only log keyed by aggregate id. That's why Event Sourcing is almost always paired with a conversion step: events are published onto a message broker (a Publish/Subscribe exchange), and a projector subscribes to that broker and builds a denormalized, query-optimized **materialized view** in a separate read database. This conversion step is exactly the CQRS split covered in Section 4.

> **Definition recap:**
>
> - **Event Store** — the append-only, ordered log of every state-changing event; the source of truth.

- **Materialized View** — a denormalized, pre-computed read database built by replaying or consuming events; optimized purely for queries, never written to directly by user commands.
- **Eventual Consistency** — the read database will reflect a write, but not necessarily the instant the write happens; there is a small, bounded delay while the event travels from store to projector.
- **Replay** — the ability to reconstruct any past state (or rebuild an entirely new read model) by re-processing the event log from the beginning.

A subtlety worth internalizing early: the event store is not "yet another database to keep in sync" — it is upstream of everything else. The read database, the UI, and any other consumer are all, architecturally, downstream projections of the event log. If you find yourself writing to the read database directly from a command handler "just this once," you've broken the model; all writes go through the event store, with no exceptions.

3. Hands-On: Building an Event-Sourced Order Service

This section builds a complete event-sourced Order aggregate from scratch, using a relational event-store table (works identically on PostgreSQL, SQL Server, or MySQL — Section 5 swaps in SQL Server specifically once we layer CQRS on top). Beyond Part I's version of this pattern, this build adds two production-grade concerns: **optimistic concurrency control** done explicitly at the application layer, and **event upcasting** for safely evolving event schemas over time.

Domain Events — Now With a Schema Version

DomainEvent.java / OrderCreatedEvent.java / ItemAddedEvent.java

```
public sealed interface DomainEvent
    permits OrderCreatedEvent, ItemAddedEvent, ItemRemovedEvent, OrderConfirmedEvent, OrderCancelledEvent {
    UUID aggregateId();
    Instant occurredAt();
    int schemaVersion();   // new: every event declares its own schema version
}

public record OrderCreatedEvent(
    UUID aggregateId, UUID customerId, Instant occurredAt, int schemaVersion
) implements DomainEvent {
    public OrderCreatedEvent(UUID aggregateId, UUID customerId, Instant occurredAt) {
        this(aggregateId, customerId, occurredAt, 2); // current schema version = 2
    }
}

public record ItemAddedEvent(
    UUID aggregateId, String productId, int quantity, BigDecimal unitPrice,
    String currency, Instant occurredAt, int schemaVersion
) implements DomainEvent {
    public ItemAddedEvent(UUID aggregateId, String productId, int quantity,
                          BigDecimal unitPrice, String currency, Instant occurredAt) {
        this(aggregateId, productId, quantity, unitPrice, currency, occurredAt, 2);
    }
}
```

> **Why `currency` is new:** imagine `ItemAddedEvent` shipped to production a year ago without a `currency` field (implicitly USD). Schema version 1 events on disk don't have this field; schema version 2 events do. The upcaster below is what lets old and new events coexist safely.

Event Upcasting — Reading Old Events With a New Schema

ItemAddedEventUpcaster.java

```
@Component
public class ItemAddedEventUpcaster {

    // Called during deserialization when a stored event's schemaVersion < current version.
    // Old v1 payloads are missing "currency" -- we backfill a safe default instead of failing to load.
    public JsonNode upcast(JsonNode rawPayload, int storedVersion) {
        if (storedVersion == 1) {
            ObjectNode node = (ObjectNode) rawPayload;
            if (!node.has("currency")) {
                node.put("currency", "USD"); // documented default at the time v1 was written
            }
            node.put("schemaVersion", 2);
            return node;
        }
        return rawPayload; // already current version, no upcasting needed
    }
}
```

EventStore.java — deserialization now runs every payload through the upcaster chain

```
@Repository
public class RelationalEventStore {
    private final JdbcTemplate jdbc;
    private final ObjectMapper mapper;
    private final ItemAddedEventUpcaster itemAddedUpcaster;

    public List<DomainEvent> loadEvents(UUID aggregateId) {
        return jdbc.query(
            "SELECT event_type, payload, schema_version FROM event_store " +
            "WHERE aggregate_id = ? ORDER BY sequence_number ASC",
            (rs, rowNum) -> deserialize(rs.getString("event_type"),
                                        rs.getString("payload"),
                                        rs.getInt("schema_version")),
            aggregateId);
    }

    private DomainEvent deserialize(String type, String payload, int storedVersion) {
```

```
        try {
            JsonNode node = mapper.readTree(payload);
            if ("ItemAddedEvent".equals(type) && storedVersion < 2)
                node = itemAddedUpcaster.upcast(node, storedVersion);
            Class<? extends DomainEvent> clazz = resolveType(type);
            return mapper.treeToValue(node, clazz);
        } catch (Exception e) { throw new EventDeserializationException(type, e); }
    }
}
```

Optimistic Concurrency — Explicit at the Application Layer

V1__event_store.sql

```
CREATE TABLE event_store (
    aggregate_id   UUID        NOT NULL,
    sequence_number BIGINT     NOT NULL,
    event_type     VARCHAR(100) NOT NULL,
    schema_version INT         NOT NULL,
    payload        TEXT        NOT NULL,
    occurred_at    TIMESTAMP   NOT NULL,
    PRIMARY KEY (aggregate_id, sequence_number)    -- the PK itself enforces one writer per version
);
```

RelationalEventStore.java — append() rejects a stale write instead of silently overwriting it

```
@Repository
public class RelationalEventStore {
    private final JdbcTemplate jdbc;

    @Transactional
    public void append(UUID aggregateId, long expectedVersion, DomainEvent event) {
        try {
            jdbc.update(
                "INSERT INTO event_store (aggregate_id, sequence_number, event_type, schema_version, payload, occurred_at) " +
                "VALUES (?, ?, ?, ?, ?, ?)",
                aggregateId, expectedVersion + 1, event.getClass().getSimpleName(),
                event.schemaVersion(), toJson(event), Timestamp.from(event.occurredAt()));
        } catch (DuplicateKeyException dup) {
            // another writer already inserted sequence_number = expectedVersion + 1 first
            throw new ConcurrentModificationException(
                "Aggregate " + aggregateId + " changed concurrently -- reload and retry the command");
        }
    }
}
```

The Aggregate — Full Lifecycle

```
public class OrderAggregate {
    private UUID id;
    private UUID customerId;
    private final Map<String, OrderLine> lines = new LinkedHashMap<>();
    private OrderStatus status = OrderStatus.DRAFT;
    private long version = 0;

    public static OrderAggregate replay(List<DomainEvent> events) {
        OrderAggregate agg = new OrderAggregate();
        for (DomainEvent e : events) { agg.apply(e); agg.version++; }
        return agg;
    }

    private void apply(DomainEvent e) {
        switch (e) {
            case OrderCreatedEvent oc -> { this.id = oc.aggregateId(); this.customerId = oc.customerId(); }
            case ItemAddedEvent ia -> lines.merge(ia.productId(),
                new OrderLine(ia.productId(), ia.quantity(), ia.unitPrice()),
                (existing, incoming) -> existing.withQuantity(existing.quantity() + incoming.quantity()));
            case ItemRemovedEvent ir -> lines.remove(ir.productId());
            case OrderConfirmedEvent ok -> this.status = OrderStatus.CONFIRMED;
            case OrderCancelledEvent oca -> this.status = OrderStatus.CANCELLED;
        }
    }

    // command handlers enforce invariants against replayed state before returning a new event
    public ItemAddedEvent addItem(String productId, int qty, BigDecimal price, String currency) {
        requireStatus(OrderStatus.DRAFT, "add items");
        if (qty <= 0) throw new IllegalArgumentException("Quantity must be positive");
        return new ItemAddedEvent(id, productId, qty, price, currency, Instant.now());
    }

    public OrderConfirmedEvent confirm() {
        requireStatus(OrderStatus.DRAFT, "confirm");
        if (lines.isEmpty()) throw new IllegalStateException("Cannot confirm an empty order");
        return new OrderConfirmedEvent(id, Instant.now());
    }
```

```
    private void requireStatus(OrderStatus required, String action) {
        if (status != required)
            throw new IllegalStateException("Cannot " + action + " while order is " + status);
    }

    public long getVersion() { return version; }
}
```

Application Service & REST Layer

```
@Service
public class OrderEventSourcedService {
    private final RelationalEventStore store;

    public UUID createOrder(UUID customerId) {
        UUID id = UUID.randomUUID();
        store.append(id, 0, new OrderCreatedEvent(id, customerId, Instant.now()));
        return id;
    }

    @Retryable(retryFor = ConcurrentModificationException.class, maxAttempts = 3)
    public void addItem(UUID orderId, String productId, int qty, BigDecimal price, String currency) {
        OrderAggregate agg = OrderAggregate.replay(store.loadEvents(orderId));
        var event = agg.addItem(productId, qty, price, currency);
        store.append(orderId, agg.getVersion(), event); // throws + retries on concurrent write
    }

    @Retryable(retryFor = ConcurrentModificationException.class, maxAttempts = 3)
    public void confirm(UUID orderId) {
        OrderAggregate agg = OrderAggregate.replay(store.loadEvents(orderId));
        store.append(orderId, agg.getVersion(), agg.confirm());
    }
}

@RestController
@RequestMapping("/orders")
public class OrderCommandController {
    private final OrderEventSourcedService service;

    @PostMapping
    public Map<String, UUID> create(@RequestBody CreateOrderRequest req) {
        return Map.of("orderId", service.createOrder(req.customerId()));
    }

    @PostMapping("/{id}/items")
    public ResponseEntity<Void> addItem(@PathVariable UUID id, @RequestBody AddItemRequest req) {
        service.addItem(id, req.productId(), req.quantity(), req.unitPrice(), req.currency());
        return ResponseEntity.accepted().build();
    }

    @PostMapping("/{id}/confirm")
    public ResponseEntity<Void> confirm(@PathVariable UUID id) {
        service.confirm(id);
        return ResponseEntity.accepted().build();
    }
}
```

Testing the Full Lifecycle, Including a Concurrency Conflict

```
# create an order
ORDER_ID=$(curl -s -X POST http://localhost:8082/orders \
  -d '{"customerId":"11111111-1111-1111-1111-111111111111"}' | jq -r .orderId)

# add two items
curl -X POST http://localhost:8082/orders/$ORDER_ID/items \
  -d '{"productId":"sku-1","quantity":2,"unitPrice":19.99,"currency":"USD"}'
curl -X POST http://localhost:8082/orders/$ORDER_ID/items \
  -d '{"productId":"sku-2","quantity":1,"unitPrice":49.99,"currency":"USD"}'

# confirm
curl -X POST http://localhost:8082/orders/$ORDER_ID/confirm

# simulate a concurrency conflict: fire two "add item" requests for the same order
# at the same instant from two terminals -- one succeeds, the @Retryable annotation
# transparently retries the other against the now-current version, and both items land correctly
```

Gotchas specific to this build:

- Upcasters must be kept forever (or until you're certain no old events remain) — deleting the v1 upcaster the moment everything looks "current" will break replay of any aggregate old enough to still contain v1 events.
- `@Retryable` on `addItem` / `confirm` assumes the retried command is idempotent to retry blindly — that's true here because the whole method reloads state and re-validates invariants on each attempt, but it would NOT be safe to retry a command that has an external side effect (like charging a card) without a separate idempotency key.
- The relational event store above uses a composite primary key `(aggregate_id, sequence_number)` specifically so the database itself — not application code — is the final arbiter of "who won" a concurrent write race.

4. CQRS + Event Sourcing + Eventual Consistency — The Combined Architecture

Putting the Three Pieces Together

Individually: CQRS separates read and write models; Event Sourcing changes how the write model persists state; Eventual Consistency describes the timing relationship between them. Combined, they form one of the most common production architectures for a high-traffic microservice: writes go through a strict, consistent relational path, while reads are served from a denormalized store built purely for query speed, kept in sync by consuming the same events the write side produces.

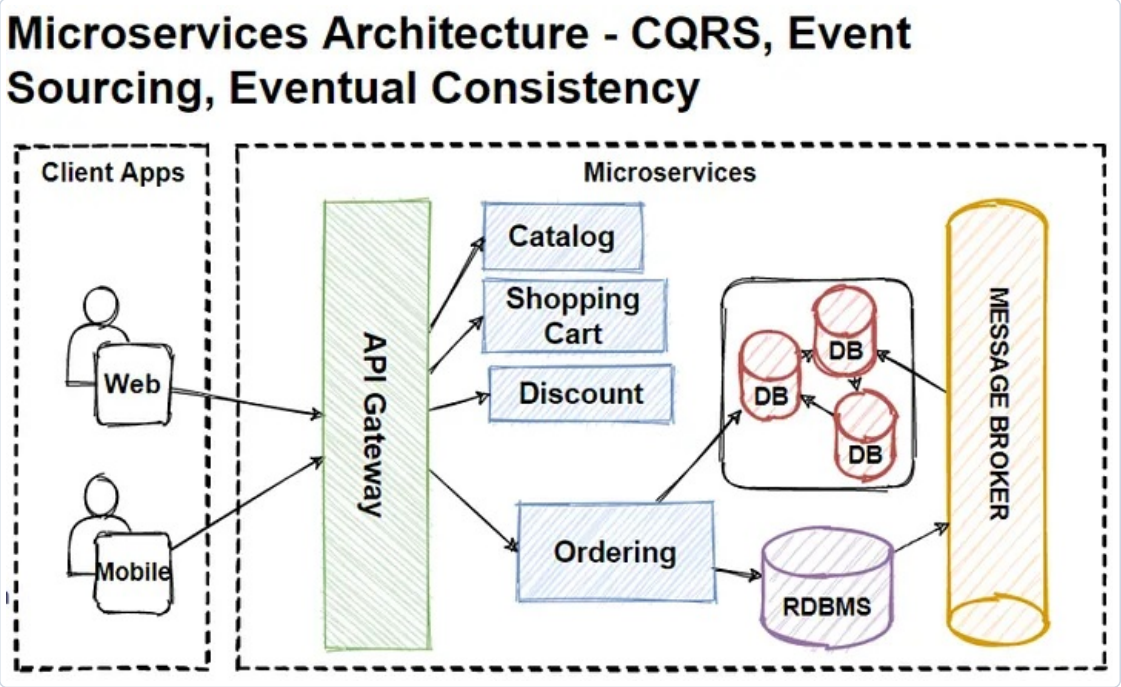


Fig. 2 — Platform view: Catalog, Shopping Cart, and Discount stay simple CRUD services, while Ordering splits into a write side (its own set of databases) and participates in a shared Message Broker that keeps a separate read side eventually consistent — all reachable through one API Gateway.

Concrete Technology Assignment for This Guide

For the Order bounded context specifically, this part of the guide assigns:

Responsibility	Technology	Why this fits
Command / write side	Microsoft SQL Server	Mature relational engine with strong ACID guarantees, well-suited to the strict validation and consistency needs of order creation/mutation
Query / read side	Apache Cassandra	Linear write scalability for high-volume order history and dashboard queries, denormalized exactly to the query shape needed
Sync mechanism	Apache Kafka (Publish/Subscribe)	Durable, replayable topic per aggregate type, guaranteeing every read-side projector eventually sees every write-side event exactly once per consumer group

The Request Flow, Step by Step

```
1. Client sends "create/update order" command
      |
      ▼
2. Command handler validates against the SQL Server-backed event-sourced aggregate (Section 3)
      |
      ▼
3. New event appended to the SQL Server event_store table (source of truth)
      |
      ▼
4. Same transaction (or transactional outbox) publishes the event to a Kafka topic
      |
      ▼
5. A Cassandra projector, subscribed to that topic, updates the read-optimized materialized view
      |
      ▼
6. Client queries "GET /orders/query/..." -> served entirely from Cassandra, no SQL Server involved
```

Defining the Eventual Consistency Window Explicitly

Rather than leaving "eventually" vague, it's worth setting an actual SLA for how stale the read side is allowed to be, and monitoring against it:

Metric	Target	How to monitor
Kafka consumer lag (read-model projector)	< 250ms p99 under normal load	Kafka consumer group lag metrics exported to Prometheus/Grafana
End-to-end command-to-queryable	< 500ms p99	Synthetic probe: issue a write, poll the read side until visible, record the delta
Read-your-writes for the initiating user	Immediate (0ms)	Route the initiating user's own follow-up read to the write side (or a short-lived cache) for a few seconds post-write

> **Read-your-writes without abandoning CQRS:** a common, pragmatic pattern is to have the command response itself return the resulting resource (as this guide's `OrderCommandController.create()` already does), so the calling client never actually needs to re-query the read side immediately after its own write — sidestepping the consistency window entirely for the common "submit and see confirmation" case.

5. Hands-On: SQL Server Write Side + Cassandra Read Side + Kafka Sync

Write-Side Setup — SQL Server

pom.xml

```
<dependency>
  <groupId>com.microsoft.sqlserver</groupId>
  <artifactId>mssql-jdbc</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.retry</groupId>
  <artifactId>spring-retry</artifactId>
</dependency>
```

docker-compose.yml addition — SQL Server for the Order write side

```
  sqlserver-order:
    image: mcr.microsoft.com/mssql/server:2022-latest
    environment:
      ACCEPT_EULA: "Y"
      SA_PASSWORD: "Ord3r_Str0ng_Pwd!"
    ports: ["1433:1433"]
```

application.yml (order-command-service)

```
spring:
  application:
    name: order-command-service
  datasource:
    url: jdbc:sqlserver://localhost:1433;databaseName=orderdb;encrypt=false
    username: sa
    password: Ord3r_Str0ng_Pwd!
    driver-class-name: com.microsoft.sqlserver.jdbc.SQLServerDriver
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      acks: all
      properties:
        enable.idempotence: true
```

The Transactional Outbox — Guaranteeing the Event Actually Gets Published

Appending to the event store and publishing to Kafka are two separate systems; a naive "insert then publish" sequence can crash between the two steps, leaving an event durably stored but never published (or vice versa). The transactional outbox pattern fixes this: the event is written to an `outbox` table in the *same SQL Server transaction* as the event store insert, and a separate poller reliably publishes from the outbox to Kafka, deleting each row only after a confirmed send.

V2__outbox.sql

```
CREATE TABLE outbox (
    id             UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    aggregate_id   UNIQUEIDENTIFIER NOT NULL,
    event_type     NVARCHAR(100)    NOT NULL,
    payload        NVARCHAR(MAX)    NOT NULL,
    created_at     DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    published      BIT              NOT NULL DEFAULT 0
);
```

RelationalEventStore.java — event_store insert + outbox insert, one transaction

```
@Repository
public class RelationalEventStore {
    private final JdbcTemplate jdbc;
    private final ObjectMapper mapper;

    @Transactional
    public void append(UUID aggregateId, long expectedVersion, DomainEvent event) {
        try {
            jdbc.update(
                "INSERT INTO event_store (aggregate_id, sequence_number, event_type, schema_version, payload, occurred_at) " +
```

```
                "VALUES (?, ?, ?, ?, ?, ?)",
                aggregateId, expectedVersion + 1, event.getClass().getSimpleName(),
                event.schemaVersion(), toJson(event), Timestamp.from(event.occurredAt()));

            // same transaction -- if this fails, the event_store insert above rolls back too
            jdbc.update(
                "INSERT INTO outbox (aggregate_id, event_type, payload) VALUES (?, ?, ?)",
                aggregateId, event.getClass().getSimpleName(), toJson(event));
        } catch (DuplicateKeyException dup) {
            throw new ConcurrentModificationException("Concurrent write on " + aggregateId);
        }
    }
}
```

OutboxPublisher.java — polls and publishes, deleting only after Kafka confirms

```
@Component
public class OutboxPublisher {
    private final JdbcTemplate jdbc;
    private final KafkaTemplate<String, String> kafkaTemplate;

    @Scheduled(fixedDelay = 200) // poll every 200ms; a CDC-based outbox (Debezium) removes even this poll
    public void publishPending() {
        List<OutboxRow> rows = jdbc.query(
            "SELECT TOP 100 id, aggregate_id, event_type, payload FROM outbox WHERE published = 0 ORDER BY created_at",
            this::mapRow);

        for (OutboxRow row : rows) {
            kafkaTemplate.send("order-events", row.aggregateId().toString(), row.payload())
                .whenComplete((result, ex) -> {
                    if (ex == null) {
                        jdbc.update("UPDATE outbox SET published = 1 WHERE id = ?", row.id());
                    } // on failure, row stays unpublished and is retried on the next poll
                });
        }
    }
}
```

Read-Side Setup — Cassandra Projector

schema.cql

```
CREATE TABLE IF NOT EXISTS orders_ks.order_summary_by_customer (
    customer_id UUID,
    created_at TIMESTAMP,
    order_id UUID,
    status TEXT,
    item_count INT,
    total_amount DECIMAL,
    PRIMARY KEY ((customer_id), created_at)
) WITH CLUSTERING ORDER BY (created_at DESC);
```

OrderProjector.java — consumes the same events the outbox published

```
@Component
public class OrderProjector {
    private final CassandraTemplate cassandraTemplate;

    @KafkaListener(topics = "order-events", groupId = "order-read-model-projector")
    public void onEvent(@Payload String rawEvent, @Header(KafkaHeaders.RECEIVED_KEY) String aggregateId) {
        JsonNode node = parse(rawEvent);
        switch (node.get("type").asText()) {
            case "OrderCreatedEvent" -> upsertSummary(aggregateId, node, "DRAFT");
            case "OrderConfirmedEvent" -> updateStatus(aggregateId, "CONFIRMED");
            case "OrderCancelledEvent" -> updateStatus(aggregateId, "CANCELLED");
            default -> { /* ItemAdded/Removed only affect totals, recomputed below */ }
        }
    }

    private void upsertSummary(String aggregateId, JsonNode node, String status) {
        OrderSummary row = new OrderSummary();
        row.setOrderId(UUID.fromString(aggregateId));
        row.setCustomerId(UUID.fromString(node.get("customerId").asText()));
        row.setCreatedAt(Instant.parse(node.get("occurredAt").asText()));
        row.setStatus(status);
        cassandraTemplate.insert(row);
    }
}
```

OrderQueryController.java — reads exclusively from Cassandra

```
@RestController
@RequestMapping("/orders/query")
public class OrderQueryController {
    private final OrderSummaryRepository repository;
```

```
    @GetMapping("/customer/{customerId}")
    public List<OrderSummary> history(@PathVariable UUID customerId) {
        return repository.findByCustomerId(customerId); // never touches SQL Server
    }
}
```

End-to-End Test: Watching Eventual Consistency Happen

```
# 1. issue a write against the SQL Server-backed command side
curl -X POST http://localhost:8082/orders -d '{"customerId":"11111111-1111-1111-1111-111111111111"}'

# 2. watch the outbox drain (should empty out within ~200-400ms)
docker exec -it sqlserver-order /opt/mssql-tools/bin/sqlcmd \
  -S localhost -U sa -P 'Ord3r_Str0ng_Pwd!' \
  -Q "SELECT COUNT(*) FROM orderdb.dbo.outbox WHERE published = 0"

# 3. query the Cassandra-backed read side -- should now reflect the new order
curl http://localhost:8083/orders/query/customer/11111111-1111-1111-1111-111111111111
```

Gotchas:

- Polling the outbox every 200ms is simple but adds up to 200ms of avoidable latency on top of network time; a CDC-based outbox reader (Debezium tailing SQL Server's CDC feature) removes the poll delay entirely and is the standard production upgrade path.
- The outbox table grows if the publisher falls behind — always keep a housekeeping job that deletes rows already marked `published = 1` past a retention window, and alert if unpublished row count grows unbounded (a sign the publisher itself is stuck).
- Cassandra's `order_summary_by_customer` table is denormalized specifically for the "list a customer's orders" query — a brand-new query pattern (e.g., "all orders over $500 today") still needs its own dedicated table, exactly as discussed for CQRS in Part I.

6. Event Hub — Definition & Deep Dive

From "a Kafka topic per service" to "one shared Event Hub"

Everything built so far uses Kafka topics scoped to a single bounded context — `order-events` belongs to Order, `product-updated` belongs to Product. That's the right starting point. But as a platform grows, more and more independent consumer types want to react to the same underlying business activity: a monitoring system wants every event for alerting, an analytics pipeline wants every event for the data lake, a new microservice wants a subset of events, and so on. Rather than wiring a new point-to-point integration for every new consumer, an **Event Hub** generalizes the message broker into a single, universal ingress and egress point that any current or future system — application, microservice, monitoring tool, analytics job — can publish to or subscribe from.

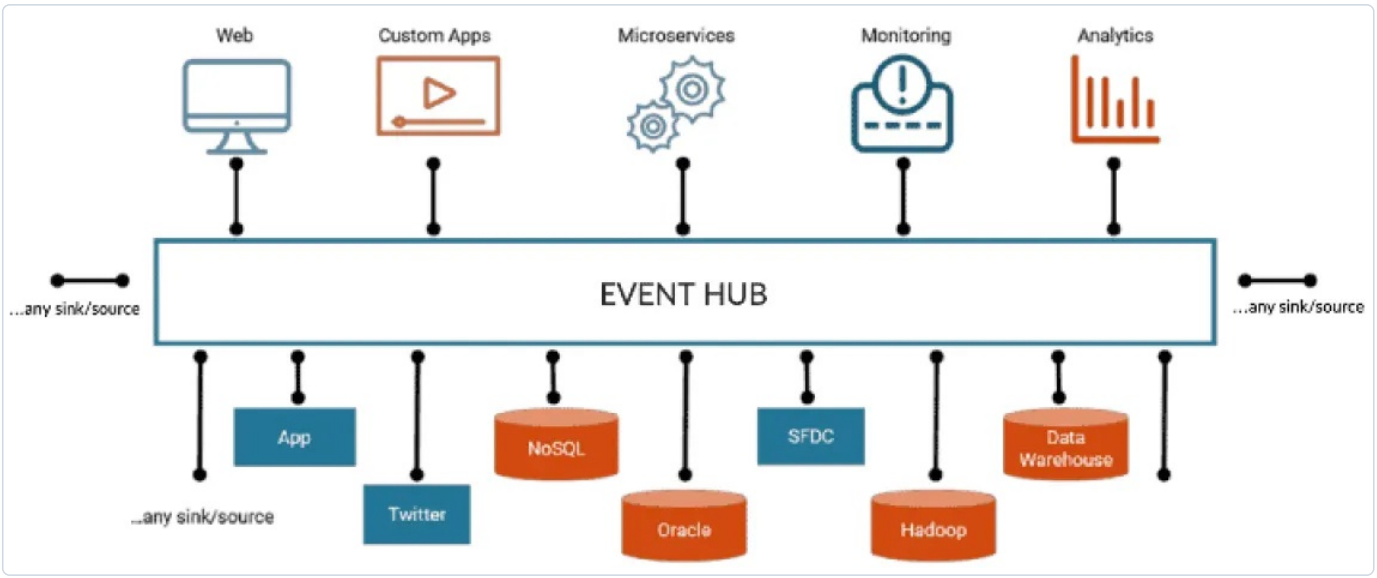


Fig. 3 — An Event Hub as the universal integration point: Web, Custom Apps, Microservices, Monitoring, and Analytics all publish and subscribe through one hub, which in turn feeds (and can be fed by) App, Twitter, NoSQL, Oracle, SFDC, Hadoop, and Data Warehouse sinks/sources — with room for "any sink/source" beyond that fixed list.

What Makes Something an Event Hub Rather Than Just "a Message Broker"

Property	Plain message broker (Part I, Bonus B)	Event Hub
Scope	Scoped to a bounded context or a handful of related services	Platform-wide; any system in the organization is a potential publisher or subscriber
Retention	Often short-lived, consumed and effectively discarded	Long retention by design — the hub doubles as a durable, replayable historical record
Consumer diversity	A small, known set of services	Deliberately open-ended: today's consumers plus any future one added without re-touching the publisher
Typical downstream use	Keeping a read model in sync (CQRS)	All of that, plus analytics/data-lake ingestion, real-time monitoring, ML feature pipelines, and ad hoc stream processing

Technology Assignment for the Event Hub Layer

Continuing this guide's stack, **Apache Kafka** plays the role of the Event Hub itself (its partitioned, durable, replayable log is exactly the right primitive), while **Apache Spark Structured Streaming** is introduced as the real-time and near-real-time processing engine that reacts to the hub's contents — covered hands-on in Section 9.

Definition recap:

- **Event Hub** — a platform-wide, durable, high-throughput event ingress/egress point that decouples every publisher from every subscriber, supporting an open-ended and evolving set of consumers.
- **Stream Processing** — continuous computation over an unbounded sequence of events as they arrive, as opposed to batch processing over a fixed, already-collected dataset.
- **Real-time vs. near-real-time** — real-time typically implies sub-second reaction latency; near-real-time tolerates seconds-to-minutes, often because the processing involves windowed aggregation.

An Event Hub is an architectural commitment, not just infrastructure. Once monitoring, analytics, and multiple microservice teams all depend on the same hub, event schemas become a de facto public contract across the whole organization — the schema-evolution discipline introduced for a single aggregate's events in Section 3 (versioning, upcasting) needs to become an organization-wide practice, often backed by a formal Schema Registry (Avro/Protobuf) rather than ad hoc JSON.

7. Hands-On: Kafka as an Event Hub for Multiple Consumer Types

Designing Topics for Platform-Wide Consumption

A single-service Kafka setup often gets away with one topic per event type. An Event Hub setup needs a little more structure so unrelated consumer types don't step on each other: a consistent topic naming convention, long retention, and schema compatibility rules enforced up front.

pom.xml

```
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
<dependency>
  <groupId>io.confluent</groupId>
  <artifactId>kafka-avro-serializer</artifactId>
  <version>7.6.0</version>
</dependency>
```

Topic creation with long retention (via Kafka admin client)

```
@Configuration
public class EventHubTopicConfig {

    @Bean
    public NewTopic orderEventsTopic() {
        return TopicBuilder.name("ecommerce.order.events")   // domain.aggregate.events naming convention
            .partitions(12)
            .replicas(3)
            .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(Duration.ofDays(90).toMillis()))
            .config(TopicConfig.CLEANUP_POLICY_CONFIG, TopicConfig.CLEANUP_POLICY_DELETE)
            .build();
    }

    @Bean
    public NewTopic productEventsTopic() {
        return TopicBuilder.name("ecommerce.product.events")
            .partitions(12).replicas(3)
            .config(TopicConfig.RETENTION_MS_CONFIG, String.valueOf(Duration.ofDays(90).toMillis()))
            .build();
    }
}
```

application.yml — Schema Registry wiring for Avro-based events

```
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: io.confluent.kafka.serializers.KafkaAvroSerializer
      properties:
        schema.registry.url: http://localhost:8081
```

One Publisher, Four Independent Consumer Types

The scenario below extends Part I's Publish-Subscribe example (Bonus B) from three application-level subscribers to four genuinely different *categories* of consumer — this is the shift from "message broker" to "Event Hub" in practice.

OrderEventPublisher.java — unchanged from the app's point of view

```
@Service
public class OrderEventPublisher {
    private final KafkaTemplate<String, OrderConfirmedEvent> kafkaTemplate;

    public void publishConfirmed(OrderConfirmedEvent event) {
        kafkaTemplate.send("ecommerce.order.events", event.aggregateId().toString(), event);
    }
}
```

Consumer 1 — Cassandra read-model projector (application concern, Section 5)

```
@Component
public class OrderReadModelProjector {
    @KafkaListener(topics = "ecommerce.order.events", groupId = "read-model-projector")
```

```
    public void onEvent(OrderConfirmedEvent e) { cassandraRepo.markConfirmed(e.aggregateId()); }
}
```

Consumer 2 — Monitoring/alerting (operational concern, owned by a different team)

```
@Component
public class OrderVolumeAlertConsumer {
    @KafkaListener(topics = "ecommerce.order.events", groupId = "monitoring-alerting")
    public void onEvent(OrderConfirmedEvent e) {
        meterRegistry.counter("orders.confirmed").increment();
        // alerting rules (e.g., "confirmed orders dropped 50% in 5 min") run against this metric elsewhere
    }
}
```

Consumer 3 — Data lake ingestion (analytics concern, batch-oriented sink)

```
@Component
public class DataLakeIngestConsumer {
    @KafkaListener(topics = "ecommerce.order.events", groupId = "data-lake-ingest")
    public void onEvent(OrderConfirmedEvent e) {
        s3Writer.appendToParquet("s3://data-lake/orders/confirmed/", e);
        // lands raw events for later batch analytics, completely decoupled from the order domain
    }
}
```

Consumer 4 — Real-time stream processing (Apache Spark, detailed in Section 9)

```
// Spark subscribes directly to the same Kafka topic as an independent, unmanaged-by-Spring consumer group --
// see Section 9 for the full Structured Streaming job
```

> **The key property being demonstrated:** all four consumers above read from the exact same topic, with zero coordination between the teams that own them. Adding Consumer 5 next quarter (say, a fraud-detection service) requires no change to `OrderEventPublisher` and no change to any existing consumer — this is the concrete payoff of treating Kafka as an Event Hub rather than a point-to-point pipe.

Testing Multi-Consumer Fan-Out

```
# confirm an order, which publishes exactly once to ecommerce.order.events
curl -X POST http://localhost:8082/orders/$ORDER_ID/confirm

# verify all four consumer groups received it independently
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe \
  --group read-model-projector
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe \
  --group monitoring-alerting
docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe \
  --group data-lake-ingest
# each group's "current offset" should advance by exactly 1, independently of the others
```

8. Event-Driven Microservices Architecture — The Full Picture

Communicating Through Events, Not Requests

Event-Driven Microservices Architecture describes a platform where services primarily communicate by producing and consuming events rather than issuing direct requests to each other. Rather than Service A calling Service B to ask "what's the current state of X," Service A instead reacts to events that already describe every change to X as it happens — the data it needs arrives continuously rather than being fetched on demand. This is the architectural style all of Part I's async/CQRS/Event Sourcing patterns and this Part's Event Hub converge into as a single, coherent whole.

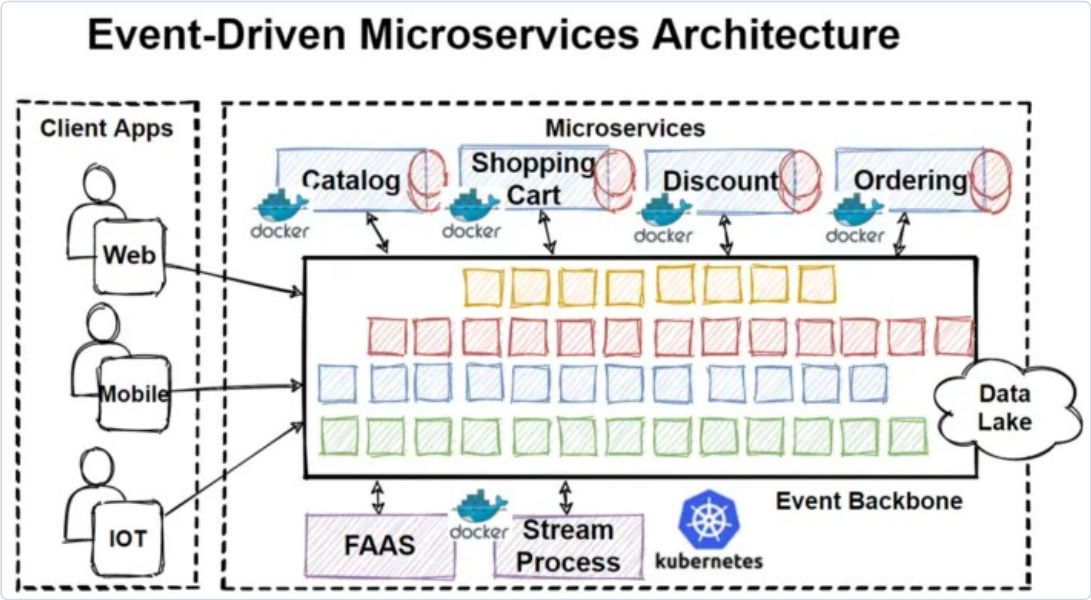


Fig. 4 — The full event-driven microservices picture: Web, Mobile, and IoT clients call Catalog, Shopping Cart, Discount, and Ordering directly, while every one of those services also publishes onto and consumes from a shared Event Backbone (colored rows), fed additionally by Stream Processing jobs and FAAS/serverless functions, with everything ultimately landing in a Data Lake.

What's New Compared to "Just Using Kafka Between Services"

- **FAAS / serverless functions** sit directly on the event backbone as lightweight, auto-scaling reactions to specific event types — no standing service needed for simple, sporadic reactions (e.g., resizing a product image whenever a `ProductImageUploaded` event fires).
- **Stream processing** (Section 9's Spark job) continuously computes rolling aggregates, joins, and windowed views directly over the event backbone, rather than only ever reacting to one event at a time.
- **The Data Lake** becomes a natural, low-effort byproduct of the architecture — since every domain event already flows through the backbone, landing a copy in cheap, durable storage for later batch analytics requires no separate change-data-capture pipeline.

Reasoning About Scale in This Architecture

Because every service in this architecture is decoupled from every other by the event backbone, scaling becomes a per-component decision rather than a whole-system one: the Catalog service can be scaled independently of the Ordering service, and the stream-processing layer can be scaled independently of either, based purely on its own load. Combined with container orchestration (Section 10), this is what allows this architecture stage to comfortably absorb the "millions of concurrent requests" row from Part I's capacity-planning table.

> **Recap — the full pattern stack that gets you here:** Database per Microservice (data isolation) → API Gateway / BFF (client entry points) → Service Aggregator (reducing chatty synchronous chains) → CQRS + Event Sourcing (Sections 2–5 of this Part) → a shared Event Hub (Sections 6–7) → stream processing and serverless reactions layered on top (Section 9) → container orchestration to scale every piece independently (Section 10). Nothing here is a single pattern — it's the accumulated result of applying the right pattern at the point the previous stage's ceiling was actually reached.

9. Hands-On: Apache Spark Structured Streaming on the Event Hub

Why Spark Alongside Kafka Consumers

The `@KafkaListener` consumers built in Section 7 are perfect for reacting to one event at a time. But questions like "how many orders were confirmed per minute, per region, over the last hour" need continuous, windowed aggregation across a stream of events — exactly what Apache Spark Structured Streaming is built for. Spark subscribes to the same Kafka topics as just another consumer group, so it fits onto the Event Hub without any special integration on the publishing side.

pom.xml (spark-streaming-job module)

```
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql_2.13</artifactId>
  <version>3.5.1</version>
</dependency>
<dependency>
  <groupId>org.apache.spark</groupId>
  <artifactId>spark-sql-kafka-0-10_2.13</artifactId>
  <version>3.5.1</version>
</dependency>
```

Job: Rolling Order-Confirmation Counts, Windowed by Minute

OrderConfirmationStreamJob.java

```
public class OrderConfirmationStreamJob {

    public static void main(String[] args) throws Exception {
        SparkSession spark = SparkSession.builder()
            .appName("order-confirmation-windowed-counts")
            .master("local[*]")    // swap for a real cluster master URL in production
            .getOrCreate();

        // 1. read the raw event hub topic as a streaming DataFrame
        Dataset<Row> rawEvents = spark.readStream()
            .format("kafka")
            .option("kafka.bootstrap.servers", "localhost:9092")
            .option("subscribe", "ecommerce.order.events")
            .option("startingOffsets", "latest")
            .load();

        // 2. parse the JSON/Avro payload and extract the event timestamp + type
        StructType eventSchema = new StructType()
            .add("aggregateId", DataTypes.StringType)
            .add("type", DataTypes.StringType)
            .add("occurredAt", DataTypes.TimestampType)
            .add("region", DataTypes.StringType);

        Dataset<Row> parsedEvents = rawEvents
            .selectExpr("CAST(value AS STRING) as json")
            .select(functions.from_json(functions.col("json"), eventSchema).as("event"))
            .select("event.*")
            .filter(functions.col("type").equalTo("OrderConfirmedEvent"));

        // 3. windowed, watermarked aggregation -- tolerates up to 2 minutes of late-arriving events
        Dataset<Row> windowedCounts = parsedEvents
            .withWatermark("occurredAt", "2 minutes")
            .groupBy(
                functions.window(functions.col("occurredAt"), "1 minute"),
                functions.col("region"))
            .count();

        // 4. sink: write results to Cassandra (the same read-side store used in Section 5)
        StreamingQuery query = windowedCounts.writeStream()
            .foreachBatch((Dataset<Row> batchDf, Long batchId) -> {
                batchDf.write()
                    .format("org.apache.spark.sql.cassandra")
                    .option("keyspace", "orders_ks")
                    .option("table", "order_confirmations_per_minute")
                    .mode(SaveMode.Append)
                    .save();
            })
            .outputMode(OutputMode.Update())
            .trigger(Trigger.ProcessingTime("30 seconds"))
            .start();

        query.awaitTermination();
    }
}
```

schema.cql — the Cassandra sink table this job writes into

```
CREATE TABLE IF NOT EXISTS orders_ks.order_confirmations_per_minute (
    window_start TIMESTAMP,
    window_end   TIMESTAMP,
    region       TEXT,
    order_count  BIGINT,
    PRIMARY KEY ((region), window_start)
) WITH CLUSTERING ORDER BY (window_start DESC);
```

Submitting the Job Alongside the Rest of the Stack

docker-compose.yml addition — a minimal single-node Spark cluster for local development

```
  spark-master:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=master
    ports: ["8090:8080", "7077:7077"]

  spark-worker:
    image: bitnami/spark:3.5.1
    environment:
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
    depends_on: [spark-master]
```

```
# package and submit the job to the local Spark cluster
mvn -pl spark-streaming-job clean package
docker cp spark-streaming-job/target/spark-streaming-job.jar spark-master:/tmp/job.jar
docker exec -it spark-master /opt/bitnami/spark/bin/spark-submit \
  --master spark://spark-master:7077 \
  --class com.example.streaming.OrderConfirmationStreamJob \
  /tmp/job.jar
```

Verifying Windowed Output

```
# confirm several orders across a couple of regions
for i in 1 2 3; do curl -X POST http://localhost:8082/orders/$ORDER_ID_1/confirm; done
curl -X POST http://localhost:8082/orders/$ORDER_ID_2/confirm

# after the next 30-second trigger interval, check the Cassandra sink table
docker exec -it cassandra cqlsh -e \
  "SELECT * FROM orders_ks.order_confirmations_per_minute WHERE region = 'us-east' LIMIT 5;"
# expect a row per 1-minute window showing the rolling confirmed-order count for that region
```

Gotchas:

- `startingOffsets = "latest"` is right for a long-running production job (don't reprocess the entire topic history on every restart) but wrong for local testing of a brand-new job — use `"earliest"` temporarily if you need to see results against events that already exist on the topic.
- Watermarking (`withWatermark`) is what allows Spark to eventually drop state for old windows — without it, the job's in-memory state grows unbounded for a topic that never stops receiving events.
- `foreachBatch` writes are at-least-once by default; if the Cassandra sink table's writes aren't naturally idempotent (as the additive counts here are, since Cassandra's overwrite-by-primary-key semantics make a retried write harmless), you'd need an explicit dedup key per micro-batch.

When to reach for Spark specifically: once a stream-processing need involves windowed aggregation, joins across multiple topics, or stateful computation over time — a plain `@KafkaListener` handles simple one-event-at-a-time reactions perfectly well and doesn't need a separate cluster to operate.

10. Capacity Planning: Concurrency, Containers & Orchestrators

Revisiting the Question: How Many Concurrent Requests Can This Design Handle?

Part I's capacity table mapped user/request volume to architecture stage in the abstract. With the full event-driven architecture from Section 8 in place, the same question has a concrete, actionable answer: because every service (Catalog, Shopping Cart, Ordering's command side, Ordering's read side, the Spark streaming job) is independently deployable and decoupled via the event backbone, each can be scaled horizontally, independently, in direct response to its own load — via a container orchestrator rather than manual provisioning.

Kubernetes Horizontal Pod Autoscaler, Tied to Real Metrics

Rather than scaling on CPU alone (a poor proxy for a service whose real bottleneck is Kafka consumer lag or request latency), this guide's order-command-service and order-query-service scale on the metrics that actually matter for each of them.

`order-command-service-hpa.yaml — scales on request latency/throughput`

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-command-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-command-service
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 65
    - type: Pods
      pods:
        metric:
          name: http_server_requests_p99_seconds   # exported via Micrometer + Prometheus adapter
        target:
          type: AverageValue
          averageValue: "500m"   # scale out once p99 latency creeps past 500ms
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30
      policies:
        - type: Percent
          value: 100
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300   # scale down cautiously to avoid flapping
```

`order-read-model-projector-hpa.yaml — scales on Kafka consumer lag, not CPU`

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: order-projector-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: order-read-model-projector
  minReplicas: 2
  maxReplicas: 12
  metrics:
    - type: External
      external:
        metric:
          name: kafka_consumergroup_lag   # exported by kafka-lag-exporter / Prometheus
          selector:
            matchLabels:
              consumergroup: read-model-projector
              topic: ecommerce.order.events
        target:
          type: AverageValue
          averageValue: "1000"   # add a replica once lag exceeds 1000 uncommitted messages per partition
```

Matching Kafka Partitions to Consumer Scale-Out

Horizontal scaling of a consumer group is capped by the partition count of the topic it reads — a 12-partition topic can support at most 12

active consumer instances in one group, with any beyond that sitting idle. Section 7's `ecommerce.order.events` topic was deliberately created with 12 partitions specifically to leave headroom for the projector's HPA above to scale up to a meaningful ceiling.

Load tier (from Part I's capacity table)	order-command-service replicas	Kafka partitions (order-events)	order-read-model-projector replicas
~500 req/s	3	6	2
~5,000 req/s	8–10	12	4–6
~50,000 req/s	20–30	24	10–12
100,000+ req/s	Multiple regional clusters, each with its own Kafka cluster mirrored via MirrorMaker 2	48+ per regional cluster	Scaled per-region, with Spark (Section 9) handling cross-region rollups

Repartitioning is disruptive — plan ahead. Increasing a Kafka topic's partition count later changes the key-to-partition mapping for all future messages (though not past ones), which can break ordering guarantees consumers were relying on. It's cheaper to over-provision partition count modestly up front (partitions are cheap; idle consumer capacity is not) than to repartition a live, business-critical topic later.

Gotchas:

- Scaling `order-command-service` without also scaling the SQL Server connection pool (or moving to read replicas / sharding) just moves the bottleneck from the application tier to the database tier — always capacity-plan the datastore alongside the service.
- The outbox poller (Section 5) becomes a scaling constraint of its own if run as a singleton — either shard it by aggregate id ranges across replicas, or migrate to a CDC-based outbox reader (Debezium) which scales naturally with Kafka Connect's own worker pool.
- Autoscaling the Spark streaming job (Section 9) is a different exercise entirely from autoscaling a stateless HTTP service — dynamic allocation needs to be configured explicitly (`spark.dynamicAllocation.enabled=true`) and tuned against the specific windowing/state size of the job.

11. Recap & Where to Go Next

What Part II Added on Top of Part I

Topic	Key addition in this Part
Event Sourcing	Event schema versioning with explicit upcasting; application-level optimistic concurrency with automatic retry
CQRS + Eventual Consistency	A named SLA for the consistency window, monitored via Kafka consumer lag and a synthetic end-to-end probe; a transactional outbox guaranteeing the write-then-publish step is atomic
Event Hub	Promoting a per-service Kafka topic into a platform-wide, long-retention, schema-registry-backed integration point serving an open-ended set of consumers
Event-Driven Architecture	The full picture including FAAS/serverless reactions and stream processing sitting directly on the event backbone, feeding a Data Lake as a natural byproduct
Stream Processing	A complete Apache Spark Structured Streaming job performing windowed, watermarked aggregation over the event hub
Capacity Planning	Concrete Kubernetes HPA configurations tied to real metrics (p99 latency, Kafka consumer lag) rather than CPU alone, plus a partition-count-to-scale table

> **The single idea underlying all of it:** once state changes are modeled as immutable, ordered events rather than in-place updates, almost everything else in this Part follows naturally — CQRS is just "read from a projection of the events," an Event Hub is just "let more things subscribe to the same events," and stream processing is just "compute continuously over the events instead of one at a time." The upfront cost is real (schema discipline, eventual consistency, more moving parts) — but so is the payoff: every piece of this architecture can be understood, deployed, and scaled independently of every other piece.

Where to Go Next

- **Back to Part I** for the ten foundational patterns this Part builds on — particularly Pattern 1 (Database per Microservice), Pattern 2 (CQRS), Pattern 3 (Event Sourcing), and Bonus B/C (Message Broker & Event-Driven Architecture) if you want the simpler, PostgreSQL/Cassandra version of these ideas before layering on SQL Server, the transactional outbox, and Spark.
- **A Schema Registry deep dive** (Avro or Protobuf with Confluent Schema Registry, briefly introduced in Section 7) is the natural next investment once more than a handful of teams depend on the same Event Hub topics.
- **Distributed tracing across the event backbone** — propagating a correlation id through Kafka message headers so a single business transaction can be traced from the initial HTTP request, through the outbox, through every Kafka consumer, and into the Spark job — is the next production-hardening step once this architecture is live.

— End of Part II — Companion to Microservice Design Patterns: Extended Hands-On Edition — July 2026 —